Introduction

This document is a post-mortem document and also aims to provide an in depth explanation of the technical details behind the code and general thought process behind the test.

I've taken 14.36h to make this project without counting some research I did before I started, here you can find a detailed breakdown of those 14h:

DESCRIPTION	DURATION	MEMBER	PROJECT	TAGS	TIME | DATE
Base UI	0:59:02	Raul Pr011...	Tech Test		6:00 PM - 6:59 PM 02/26/2026
Base UI	0:26:52	Raul Pr011...	Tech Test		5:05 PM - 5:32 PM 02/26/2026
UI Sprite Blurring	0:42:00	Raul Pr011...	Tech Test		12:55 PM - 1:37 PM 03/06/2026
Settings Conection and code manager	0:31:31	Raul Pr011...	Tech Test		7:43 PM - 8:15 PM 03/05/2026
Level Complete UI	1:00:13	Raul Pr011...	Tech Test		5:57 PM - 6:57 PM 03/05/2026
Level Complete UI	1:03:25	Raul Pr011...	Tech Test		1:46 PM - 2:50 PM 03/05/2026
Level Complete UI	0:58:17	Raul Pr011...	Tech Test		10:36 PM - 11:34 PM 03/04/2026
Level Complete UI	0:25:42	Raul Pr011...	Tech Test		9:51 PM - 10:17 PM 03/04/2026
Settings Screen UI	0:29:00	Raul Pr011...	Tech Test		8:10 PM - 8:39 PM 03/04/2026
Toggle Settings Tweening	0:35:59	Raul Pr011...	Tech Test		7:04 PM - 7:40 PM 03/04/2026
Settings Screen UI	0:34:33	Raul Pr011...	Tech Test		1:05 PM - 1:40 PM 03/04/2026
Settings Screen UI	0:00:06	Raul Pr011...	Tech Test		1:05 PM - 1:05 PM 03/04/2026
Settings Screen UI	0:10:49	Raul Pr011...	Tech Test		12:25 PM - 12:36 PM 03/04/2026
Settings Screen UI	0:36:13	Raul Pr011...	Tech Test		5:53 PM - 6:29 PM 03/03/2026
Bottom Bar Button Logic	0:51:02	Raul Pr011...	Tech Test		5:19 PM - 6:10 PM 03/02/2026
Bottom Bar Button Logic	0:19:23	Raul Pr011...	Tech Test		1:07 PM - 1:27 PM 03/02/2026
Sparkles and VFX	0:37:46	Raul Pr011...	Tech Test		7:23 PM - 8:01 PM 03/06/2026
Unity UI Sprite Blurring Shader	1:03:02	Raul Pr011...	Tech Test		5:28 PM - 6:31 PM 03/06/2026
Unity UI Sprite Blurring Shader	0:25:12	Raul Pr011...	Tech Test		1:37 PM - 2:02 PM 03/06/2026
Hide Button Bar	0:20:51	Raul Pr011...	Tech Test		8:45 PM - 9:06 PM 03/09/2026
Confeti VFX	0:24:38	Raul Pr011...	Tech Test		8:11 PM - 8:36 PM 03/09/2026
Final Settings Programming	0:25:13	Raul Pr011...	Tech Test		7:08 PM - 7:33 PM 03/09/2026
Final Settings Programming	1:03:37	Raul Pr011...	Tech Test		5:32 PM - 6:35 PM 03/09/2026
Shader Polish	0:09:08	Raul Pr011...	Tech Test		2:27 PM - 2:36 PM 03/09/2026
Shader Polish	0:07:52	Raul Pr011...	Tech Test		2:00 PM - 2:08 PM 03/09/2026

Index

- General Workflow on UI Import
  - Unity PSD Importer
  - Individual export of PNG Assets
  - Texture Packer - External Software
- Scripts
  - NotchDetector
  - TweeningAnimation
  - BottomBarView
  - BottomBarButton
  - GenericToggle
  - MainSceneController
- Shaders
  - SH_DiagonalOpening
  - SH_UIBlur
  - SH_LevelCompleteBG
  - SH_AnimatedGlow
  - TMP_SDF_AlphaMask
- Notes about VFX

General Workflow on UI Import

In order to import and prepare the UI for Unity based on the .PSD files provided I've used different techniques:

Unity PSD Importer

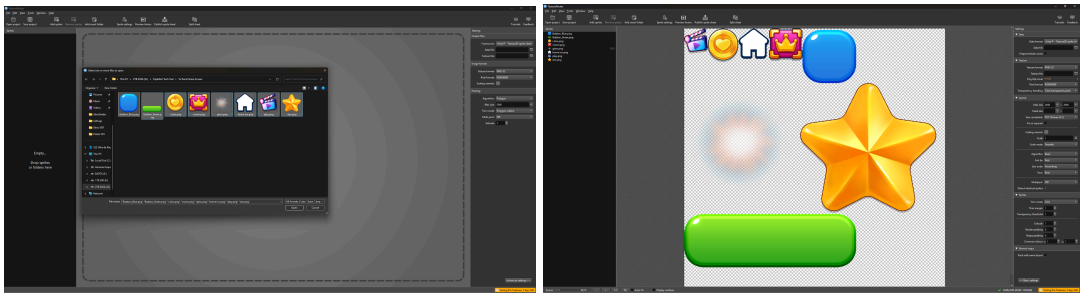
Unity's PSD Importer package does some heavy lifting to import a bunch of assets, it presents some problems though.

- It's fast and easy to use but…
  - **Avoid having your background in the PSD file, as it will pack it together with the rest of the assets,** making the power of 2 sprite atlas gigantic in order to have good pixel density. *I know that I did this with the home UI, that's why I'm noting this down.*
  - **Some layer effects don't export properly, make sure to rasterize all layers.**

Individual export of PNG Assets

For simpler things I've just exported them by rasterizing the layer effects on Photoshop and then exporting individual layers using the corresponding Photoshop option on the layer contextual menu.

Texture Packer - External Software



This is probably the best option, it automatically packs every sprite together on a single power of 2 spritesheet and generates a data file that with the Unity package provided can be read to automatically trim every asset. The only con is that it's a paid tool, useful if you work daily on 2D Assets. This was used for the level complete UI!

Scripts

Find here an in depth explanation behind the scripts of the project. To see the full code please use the github repo, as I plan to only show some relevant snippets.

NotchDetector

```
private void Start()
{
    rectTransform = GetComponent<RectTransform> ();
    float aspect = (Screen.currentResolution.height / Screen.safeArea.height);
    if (aspect > 1.03f)
    {
        rectTransform.anchoredPosition = posWithNotch;
    }
    else
    {
        rectTransform.anchoredPosition = posWithoutNotch;
    }
}
```

A simple script that computes a ratio based on the safe area height and current resolution. Within a certain threshold it takes it as a notch is present in the device and adjusts the vertical position of the top buttons in the main screen.

TweeningAnimation

```
    public void UniformScale(float EndScale, float duration)
    {
    }
    public void RotateLocked()
    {
    }
    public void OnPointerEnter(PointerEventData eventData)
    {
        Debug.Log("Pointer Enter");
        switch (mode)
        {
            case TweeningModes.UniformScale:
                UniformScale(uniformEndScale, uniformScaleDuration);
                break;
            case TweeningModes.Rotate:
                //RotateLocked();
                break;
            default:
                break;
        }
    }
```

A modular script with a custom editor that provides different types of tweening animation when the cursor hovers / touches an UI Element. The idea is that with the TweeningModes enum anyone can implement their own type of tweening behaviour by adding more options.

One can also implement their own behaviour by creating void methods that can be invoked using a OnClick() event in a button. RotateLocked() is an example used in the locked options of the bottom navigation bar.

BottomBarView

```
public class BottomBarView : MonoBehaviour
{
    public bool IsSomethingSelected()
    {
        if (buttons != null)
        {
            for (int i = 0; i < buttons.Count; i++)
            {
                if (buttons[i].buttonEnabled)
                {
                    return true;
                }
            }
            return false;
        }
        else
        {
            return false;
        }
    }
    public void CheckForNoContent()
    {
    }
    public void Subscribe(BottomBarButton button)
    {
    }
    public void Unsubscribe(BottomBarButton button)
    {
    }
    public void CloseOtherEnabledButtons(bool silent)
    {
        for (int i = 0; i < buttons.Count; i++)
        {
            if (buttons[i].buttonEnabled)
            {
                buttons[i].CloseButton(silent);
            }
        }
    }
    public void HideFooter()
    {
    }

}
```

A script that handles global actions for the footer on the main UI. It lists the buttons on the bottom bar, provides logic and helper functions to the buttons like checking information about other buttons, closing all the buttons and hiding the bottom bar. It also holds the ContentActivated() and Closed() methods.

BottomBarButton

```
    public void CloseButton(bool silent)
    {
        buttonEnabled = false;
        rectTransform.DOSizeDelta(new Vector2(0f, 667.457f), 0.3f);
        childRectTransform.DOLocalMoveY(0f, 0.3f);
        image.DOFade(0, 0.3f).SetEase(fadeCurve);
        textMesh.DOFade(0, 0.3f).SetEase(fadeCurve);
        textMesh.DOScale(0, 0.3f).SetEase(fadeCurve);

        if (!silent)buttonController.CheckForNoContent();

    }

    public void OpenButton()
    {
        if (locked == false)
        {
            buttonController.CloseOtherEnabledButtons(true);
            buttonEnabled = true;

            //Do Tweening Stuff
            rectTransform.DOSizeDelta(new Vector2(946.8f, 667.457f), 0.3f);
            childRectTransform.DOLocalMoveY(190.49f, 0.3f);
            image.DOFade(1, 0.3f).SetEase(fadeCurve);
            textMesh.DOFade(1,0.3f).SetEase(fadeCurve);
            textMesh.DOScale(1,0.3f).SetEase(fadeCurve);
            //

            buttonController.ContentActivated();

        }
        else
        {
            RotateLocked();
        }

    }
```

A script that handles the animation and logic behind opening and closing the bottom buttons, like everything in the UI it uses the DOTween library. Also it calls to the content check in ButtonBar if the silent parameter is false, this is to avoid calling the method that checks if there's no content active when swapping between different buttons, as this should only be checked when the player actively closes a button and not when the method that forces other buttons to close is fired.

GenericToggle

```
public class GenericToggle : MonoBehaviour
{
    public Image background;
    public RectTransform dot;

    public Color deactivatedColor;
    public Color activeColor;

    public float stateTimeTransition = 0.3f;

    public void TweenBasedOnState(bool state)
    {
        //Debug.Log(state);
        if (state == true)
        {
            background.DOColor(deactivatedColor, stateTimeTransition);
            dot.DOAnchorPos(new Vector2(0f, 5.499924f),
stateTimeTransition);
        }
        else
        {
            background.DOColor(activeColor, stateTimeTransition);
            dot.DOAnchorPos(new Vector2(-194f, 5.499924f),
stateTimeTransition);
        }
    }
}
```

A script that handles animation of the toggles in the settings panel. Unity has a native toggle component for UI but it works like an on/off checkmark and does not feature the modern design standard of toggle design present in touch user interfaces.

MainSceneController

```
IEnumerator RecordFrame()
{
}

public void OpenUISettingsPanel()
{
}

public void OpenSettings()
{
}

public void ClosePopUp()
{
}

public void CloseSettings()
{
}

IEnumerator RevealLevelCompleteUI()
{
}

IEnumerator HideLevelCompleteUI()
{
}

void UpdateLevelCompleteShaderProperty()
{
}

public void CompleteLevel()
{
}

public void ReturnToHomeScreen()
{
}
```

The script that handles opening and closing other parts of the UI. Multiple coroutines are used to transition in or out of the level complete screen by tweening a float that is then passed as a global shader parameter for the whole level complete UI and to give control to the different screens.

OpenSettings() and CloseSettings() handle closing and opening the settings panel and also give out control of the main UI to give it to the settings panel, it also calls a coroutine that takes a screenshot of the UI before opening the settings panel and is assigned to the background of the settings UI with a special shader that blurs the generated image, the blur float parameter being animated using a tween.

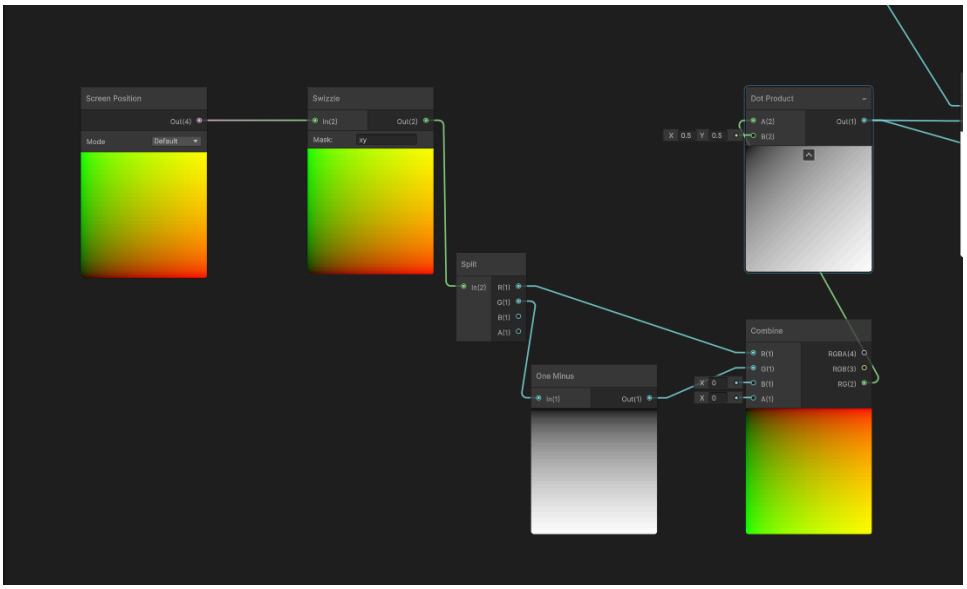
Shaders

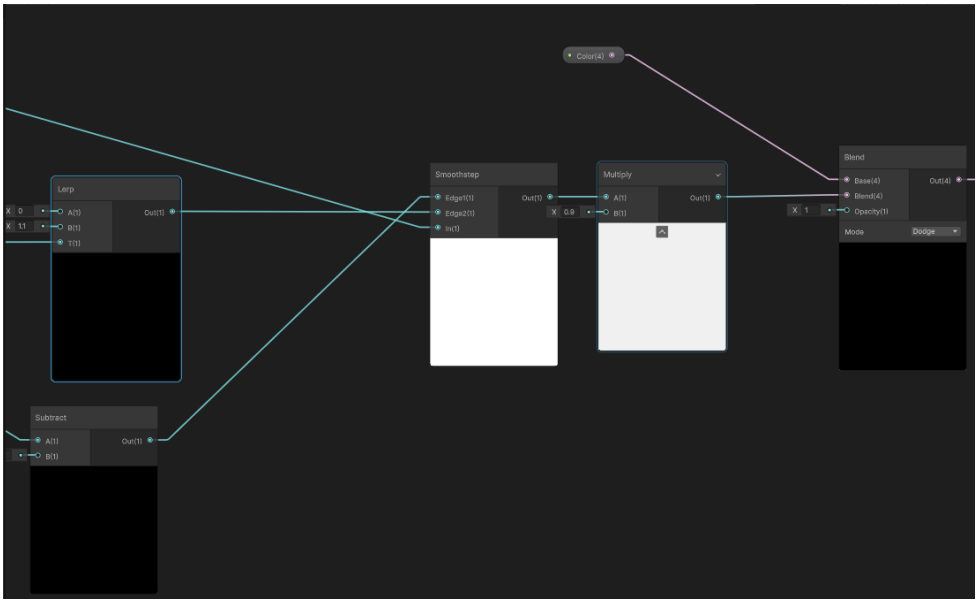
I've decided to create the shaders for the project using shadergraph and some custom function nodes.

My philosophy when creating shaders is that if it's something simple that doesn't need to use any of the URP features like custom extra passes or aren't complex shaders (like UI Canvas shaders) then work with shadergraph since on the long run it will be much simpler to maintain to those that work on VFX or simply don't have that much HLSL experience.

Shaders within a certain complexity need to be done HLSL, as I feel I can read and manipulate them faster when creating larger shaders, as large shadergraph shaders are a mess of tangled nodes even when using subgraphs. HLSL shaders also allow extra passes and a ton of extra features not yet available in Shadergraph and give me total control over some optimizations.

SH_DiagonalOpening



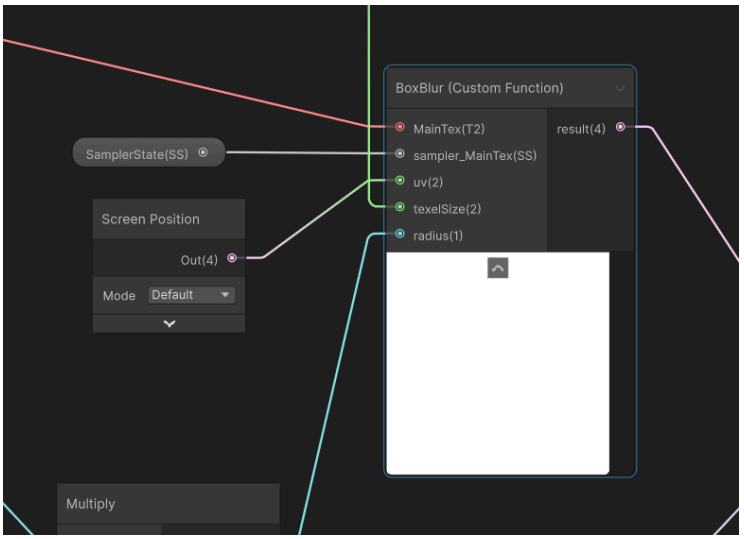


A shader used for the UI in the main screen that reveals the sprites using a diagonal mask generated using normalized device screen coordinates.

The diagonal gradient is generated by using screen coords, inverting the Y to have the gradient start at the top-left and then doing a dot product between the coordinates and (0.5,0.5).

The mask then is created by using a step function with the gradient as input and the _Reveal global parameter as the threshold. The gradient shading in the color is created by using a smoothstep with the _Reveal parameter and then blending it with the color dodge blending node.

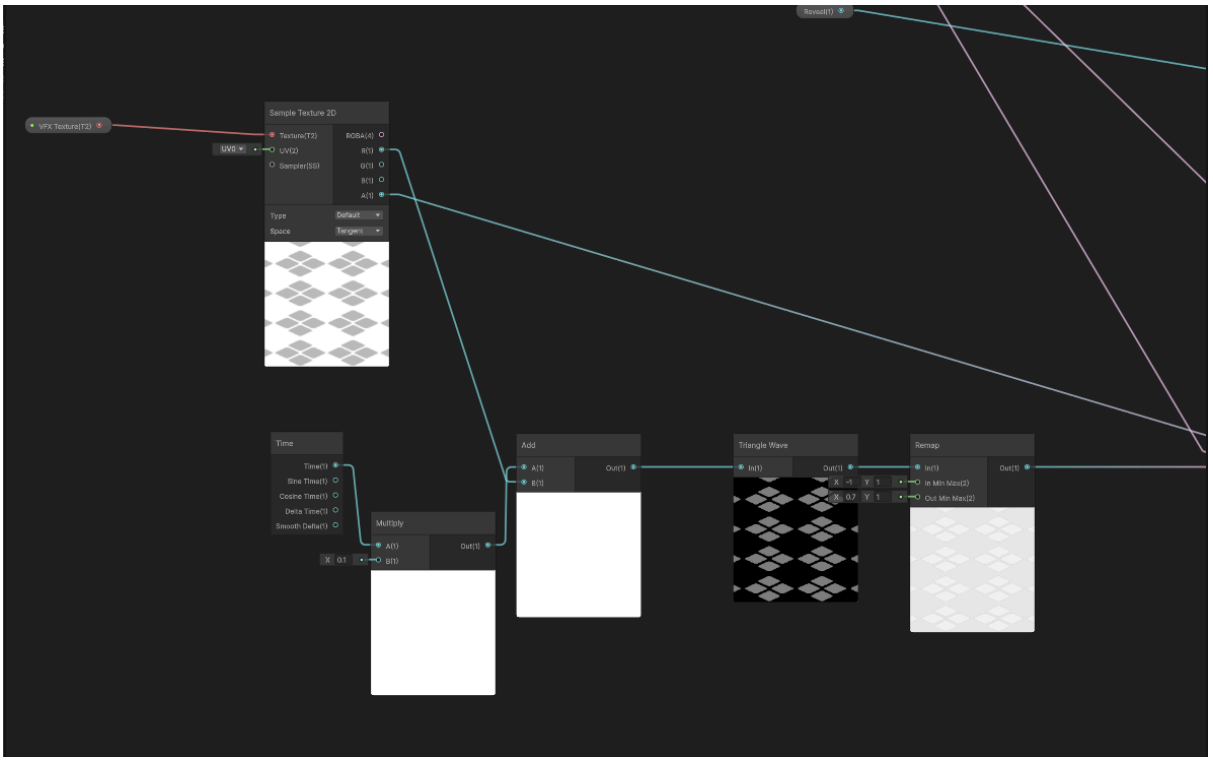
SH_UIBlur




```
for (int y = -radius; y <= radius; y++)
        {
            for (int x = -radius; x <= radius; x++)
            {
                half2 pixelOffset = half2(x,y);
                half distanceToPixel = length(pixelOffset);

                if(distanceToPixel > float(radius))
                {
                    continue;
                }

                half2 uvOffset = pixelOffset * texelSize;

                result += SAMPLE_TEXTURE2D(MainTex, sampler_MainTex, uv + uvOffset);
                numberOfSamples++;

            }
        }

        result /= numberOfSamples;
```

A simple shader that takes a texture as an input and samples it doing a box blur with some optimizations to it. Instead of doing a box we calculate if it's within the max blur circle radius and if it's outside the circle it leaves the sampling loop. This gives a more pleasant look than a regular box blur and it also saves a lot in sampling overhead because we are not doing the complete loop thanks to sampling in a circle radius and not a square.

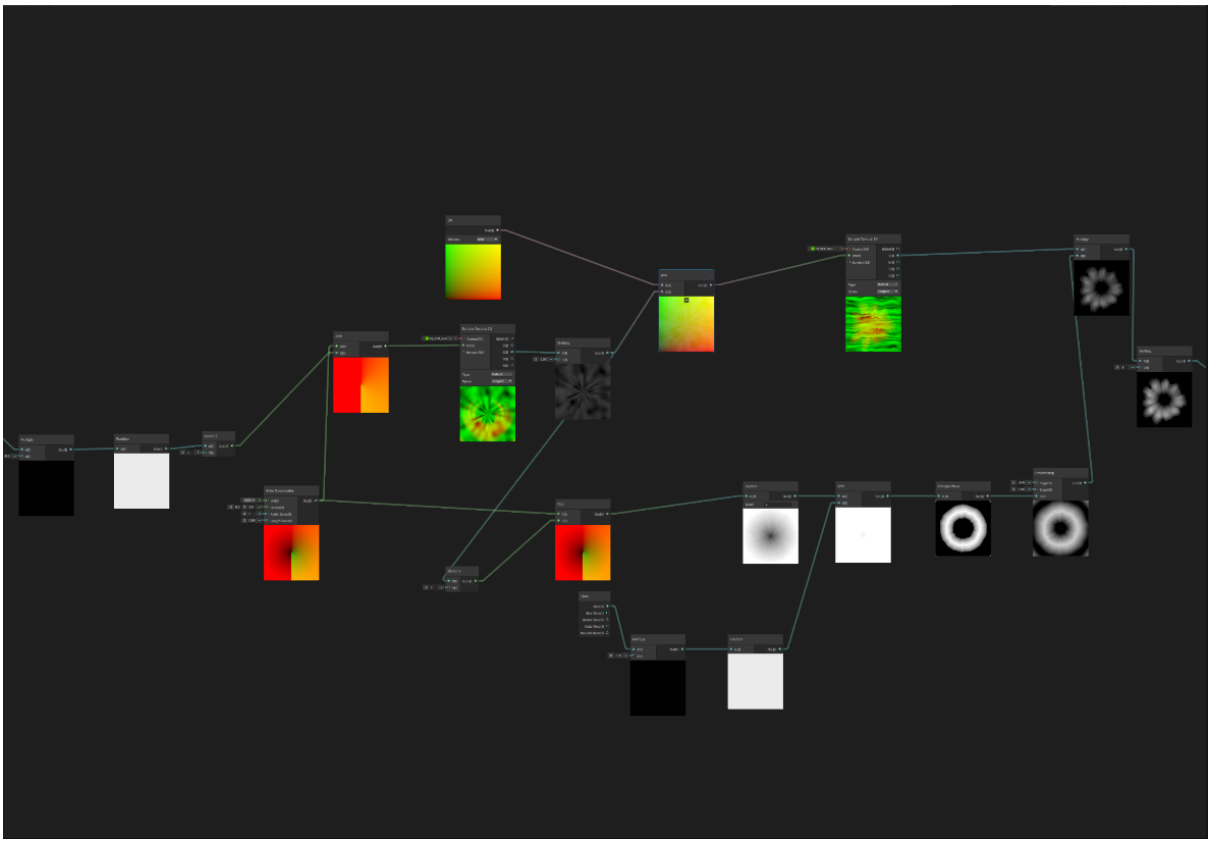
SH_LevelCompleteBG



The shader used for the background in the level complete screen. It samples the background sprite and with a special texture creates the animation effect.

The animation works by using the game time as an input for a Triangle Wave, the game time is offsetted by the grayscale value on the texture to make the animation not uniform on all of the shapes. The texture was done manually in Photoshop using the background file as source.

SH_AnimatedGlow



A simple shader used in a glow behind the star in the level complete screen. It's just some rays that get modulated by a triangle wave using polar coordinates, the rays also get displaced by a noise animation sampled again with polar coordinates.

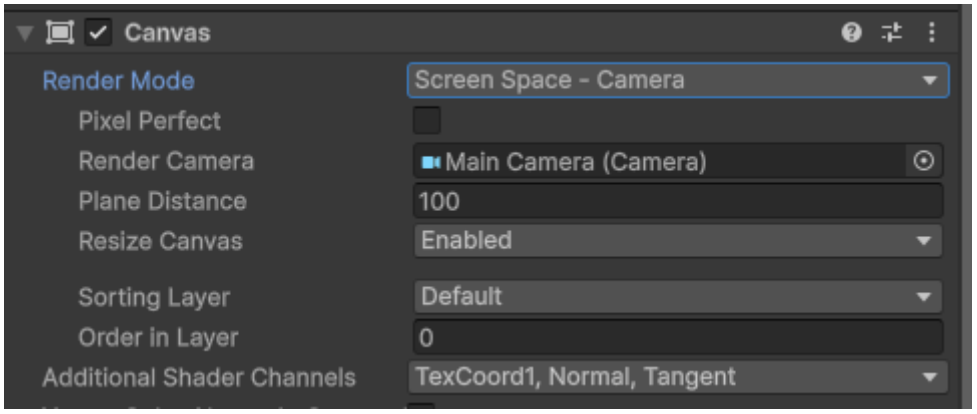
TMP_SDF_AlphaMask

```
    #if UNDERLAY_INNER
    float d = tex2D(_MainTex, input.texcoord2.xy).a * input.texcoord2.z;
    faceColor += input.underlayColor * (1 - saturate(d - input.texcoord2.w)) * saturate(1 - sd) * (1 - faceColor.a);
    #endif

    #if GLOW_ON
    float4 glowColor = GetGlowColor(sd, scale);
    faceColor.rgb += glowColor.rgb * glowColor.a;
    #endif

// Alternative implementation to UnityGet2DClipping with support for softness.
    #if UNITY_UI_CLIP_RECT
    half2 m = saturate((_ClipRect.zw - _ClipRect.xy - abs(input.mask.xy)) * input.mask.zw);
    faceColor *= m.x * m.y;
    #endif

    //INSERTED CODE TO REVEAL TEXT WITH GLOBAL PARAMETER
    float2 screenUV = (input.positionCS.xy / input.positionCS.w) * 0.5 + 0.5;
    // float2 screenUV = (input.positionCS.xy)* 0.5 + 0.5;
    // screenUV = screenUV + 0.5;
    screenUV = float2(screenUV.x, 1-screenUV.y);
    half revealValue = lerp(0,1.1, _Reveal);
    half grad = dot(half2(.5,.5), screenUV);

    half mask = 1-step(_Reveal,grad);
    half gradient = smoothstep(revealValue - 0.1, revealValue, grad) * 1;
    faceColor.rgb = Unity_Blend_Dodge_float4(faceColor.rgba, gradient, 1).rgb;
    faceColor.a *= mask;

    #if UNITY_UI_ALPHACLIP
    clip(faceColor.a - 0.001);
    #endif
```

A modified TextMesh Pro SDF shader that implements the diagonal mask and color dodge blending. This is to have consistency with the other sprites of the level complete screen.

Notes about VFX

Normally, the way you implement particles on the UI is by changing the Render Mode in the Canvas to Screen Space - Camera.



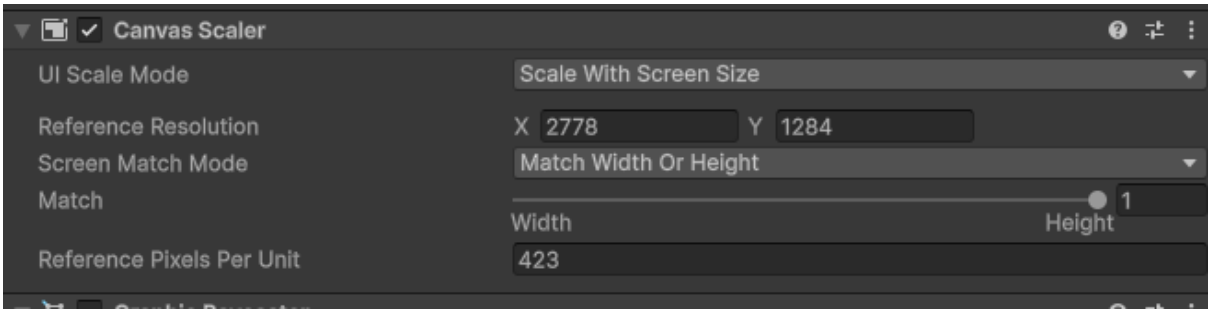
But in this case I had to rely on a script of the Unity UI Extensions Package that one can find in GitHub. Why? Because somehow normalized device coordinates either present a bug or a weird quirk in the Text Mesh Pro shaders when in Screen Space - Camera mode.

That meant that the diagonal mask on the text did not work properly and was desynchronized to the rest of sprites.

How it looks when in Screen Space - Camera mode vs Screen Space - Overlay

Improvements and suggestions

Canvas Scaling



When doing the canvas for the main UI I completely forgot of putting the Match slider closer to height than width, that means that the UI on the main screen doesn't work on a regular 16:9 screen in landscape mode, as I didn't want this to take longer that it already did I decided to leave it be as it works on what would be a tablet device, which is enough for now.

Settings and Level Complete UI are correctly set up, as those are much simpler UI's in comparison.

Sprite Packing

Sprites need to be on a power of 2 texture atlas in order to be compressed, the Unity PSD importer already does this, but ideally it's best to do it manually or leave it to an external tool like TexturePacker.